

Attention: Le rapport doit être réalisé par l'étudiant(e).

Si le rapport résulte d'une collaboration, elle doit être clairement annoncée.

NOM : CHAUDAGNE	Prénoms : Paul
Classe : 3G2	
Lycée : Lycée du Parc	Numéro de candidat : 32452
Ville : Lyon	

Concours auxquels vous êtes admissible (les indiquer par une croix) :

Banque MP		Banque MPI	
ENS Paris-Saclay	MP - Option MP MP - Option MI	ENS Paris-Saclay	Option Math Option Info
ENS de Lyon	Math - Option M-MP Math - Option M-MI Informatique	ENS de Lyon	Math MPI Info MPI
ENS Rennes	MP - Option MP MP - Option MI	ENS Rennes	Info MPI
ENS Paris	MP - Option P MP - Option I	ENS Paris	Option Math Option Info

Matière dominante du TIPE (la sélectionner d'une croix inscrite dans la case correspondante) :

Informatique	Mathématiques	Physique
--------------	---------------	----------

Titre du TIPE : Courbes elliptiques et factorisation par la méthode de Lenstra.

Nombre de pages (à indiquer dans les cases ci-dessous) :

Texte	Illustration	Bibliographie
-------	--------------	---------------

Attention, les illustrations doivent figurer dans le corps du texte et non en fin du document !

Résumé ou descriptif succinct du TIPE (6 lignes, maximum) :

Utilisation des propriétés des courbes elliptiques définies sur des corps finis afin de factoriser des entiers ayant de grands facteurs premiers de manière efficace grâce à l'algorithme probabiliste de Lenstra.

À Lyon	Signature du professeur responsable de la classe préparatoire dans la discipline	Cachet de l'établissement
Le 05/06/2026		
Signature du (de la) candidat(e)		
La signature du professeur responsable et le tampon de l'établissement ne sont pas indispensables pour les candidats libres (hors CPGE).		

LYCEE DU PARC

1, Boulevard Anatole France

69006 LYON

Tél. 04 37 51 15 51 - INE 0590026d

TIPE - Courbes elliptiques et factorisation

Paul Chaudagne (n° 32452)

Nous allons dans ce TIPE définir des ensembles que l'on munira d'une structure de groupes, appelés courbes elliptiques. Nous utiliserons ensuite ces courbes elliptiques afin de décomposer en produit de facteurs premiers des nombres composés.

I - Les courbes elliptiques

1) Définition des courbes elliptiques

Définition 1 (Plan projectif)

Soit $\mathbb{K}$ un corps. On définit la relation d'équivalence $\sim$ sur $\mathbb{K}^3 \setminus \{(0, 0, 0)\}$ par $(a_1, a_2, a_3) \sim (a'_1, a'_2, a'_3)$ si et seulement s'il existe $\lambda \in \mathbb{K} \setminus \{0\}$ tel que :

$$(a_1, a_2, a_3) = \lambda(a'_1, a'_2, a'_3)$$

On appelle **plan projectif** l'ensemble des classes d'équivalence pour $\sim$, noté :

$$\mathbb{P}^2(\mathbb{K}) := (\mathbb{K}^3 \setminus \{(0, 0, 0)\}) / \sim$$

Pour $P = (x, y, z) \in \mathbb{K}^3 \setminus \{(0, 0, 0)\}$, on notera $[x : y : z]$ la **classe d'équivalence de P** pour $\sim$.

Cela revient à projeter l'espace sur une demi-sphère centrée en $(0, 0)$, où chaque classe d'équivalence correspond à une droite passant par l'origine et un unique point de la demi-sphère, soit en dimension 1 :

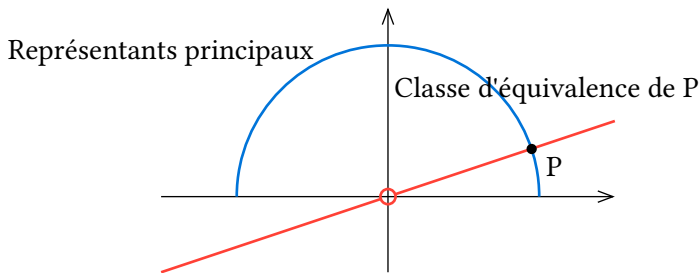


Fig. 1. – Espace projectif de dimension 1.

Définition 2 (Polynôme homogène)

Un **polynôme homogène** est un polynôme en plusieurs indéterminées dont tous les monômes non nuls sont de même degré total.

Nous ne travaillerons, par la suite qu'avec des polynômes de degré 3 homogènes en trois indéterminées, de la forme :

$$aX^3 + bY^3 + cZ^3 + dX^2Y + eX^2Z + fY^2X + gY^2Z + hZ^2X + iZ^2Y + jXYZ$$

On remarque en particulier que si P est un polynôme homogène et que $P(x, y, z) = 0$, alors :

$$\forall (x', y', z') \in [x : y : z], P(x', y', z') = \lambda^3 P(x, y, z) = 0$$

Dans le plan projectif, l'annulation d'un polynôme homogène ne dépend donc pas du représentant choisi.

Définition 3 (Cubique)

On appelle **cubique sur un corps $\mathbb{K}$** , l'ensemble des solutions dans le plan projectif $\mathbb{P}^2(\mathbb{K})$ de l'équation $F(X, Y, Z) = 0$, où F est un polynôme homogène de degré 3 en trois indéterminées à coefficients dans $\mathbb{K}$.

Formellement, pour F polynôme homogène de $\mathbb{K}_3[X, Y, Z]$, on note :

$$V := \{[x : y : z] \in \mathbb{P}^2(\mathbb{K}), F(x, y, z) = 0\}$$

On notera que multiplier le polynôme par un scalaire non nul ne change pas la cubique considérée.

Définition 4 (Singularité)

Un point $P = [x : y : z]$ d'une cubique est dit **singulier** lorsque :

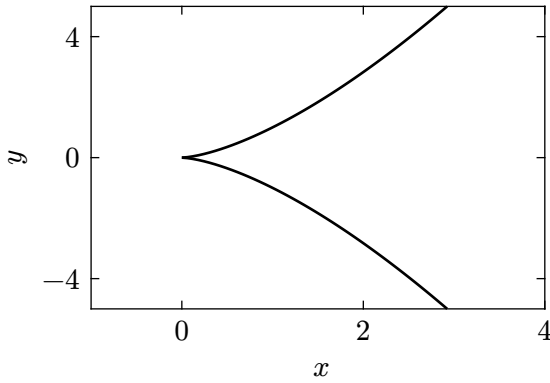
$$\left(\frac{\partial F}{\partial X}(P), \frac{\partial F}{\partial Y}(P), \frac{\partial F}{\partial Z}(P) \right) = (0, 0, 0)$$

On dira d'une cubique qu'elle est **lisse** (ou **non singulière**) si elle ne possède aucun point singulier, soit :

$$\forall P \in E(\mathbb{K}), \left(\frac{\partial F}{\partial X}(P), \frac{\partial F}{\partial Y}(P), \frac{\partial F}{\partial Z}(P) \right) \neq (0, 0, 0)$$

Définition 5 (Courbe elliptique)

On appelle **courbe elliptique** une cubique lisse, que l'on notera $E(\mathbb{K})$ ou simplement E en l'absence d'ambiguïté.

Fig. 2. – Cubique singulière définie par $y^2 = x^3$.

Par la suite, nous ne considérerons que des courbes elliptiques non singulières définies sur un corps $\mathbb{K}$ de caractéristique différente de 2 ou 3.

Définition 6 (Tangente)

Soit E une courbe elliptique, et $P \in E$ un point. Alors la **tangente à E en P** est donnée par :

$$\frac{\partial F}{\partial X}(P)(X - X_P) + \frac{\partial F}{\partial Y}(P)(Y - Y_P) + \frac{\partial F}{\partial Z}(P)(Z - Z_P) = 0$$

♣

Définition 7 (Point d'inflexion)

Soit E une courbe elliptique, on dit qu'un point $P \in E$ est un **point d'inflexion** si la tangente à E en P coupe E en P avec une multiplicité égale à 3.

♣

2) Forme de Weierstrass

Lemme 8

Une courbe elliptique possède au moins un point d'inflexion.

♥

Proposition 9

Soient E une courbe elliptique sur $\mathbb{K}$ et P un point d'inflexion de E .

Alors un changement de variables linéaire inversible permet de transformer P en $[0 : 1 : 0]$ et la tangente en P en $Z = 0$.

♣

Démonstration. Soit L la tangente à E en $P = [x_P : y_P : z_P]$. Soit $Q = [x_Q : y_Q : z_Q] \in L \setminus E$, les vecteurs (x_P, y_P, z_P) et (x_Q, y_Q, z_Q) sont linéairement indépendants car P et Q correspondent à deux points distincts du plan projectif.

On complète avec un vecteur C en une base de $\mathbb{K}^3$. On obtient alors :

$$M = \begin{pmatrix} x_Q & x_P & C \\ y_Q & y_P & C \\ z_Q & z_P & C \end{pmatrix}$$

M est inversible. M^{-1} envoie P sur le point $[0 : 1 : 0]$ et Q sur le point $[1 : 0 : 0]$. Donc M^{-1} envoie L sur $Z = 0$ car c'est l'unique droite passant par P et Q . $\square$

On peut donc se contenter d'étudier ce cas particulier dans la suite, et en déduire une étude générale.

Théorème 10 (Mise sous forme de Weierstrass)

Soit E une courbe elliptique. Soit $\mathcal{O}$ un point d'inflexion de E , si $\mathcal{O} = [0 : 1 : 0]$ et si la tangente à E en $\mathcal{O}$ est $Z = 0$, alors E est de la forme :

$$Y^2Z + a_1YXZ + a_3YZ^2 - (X^3 + a_2X^2Z + a_4XZ^2 + a_6Z^3) = 0$$

♥

Démonstration. La forme générale du polynôme qui définit une courbe elliptique est :

$$F(X, Y, Z) = aX^3 + bY^3 + cZ^3 + dX^2Y + eX^2Z + fY^2X + gY^2Z + hZ^2X + iZ^2Y + jXYZ$$

Montrons que certains termes s'annulent :

- $F([0 : 1 : 0]) = 0$ donc $b = 0$.
- La tangente à E en $[0 : 1 : 0]$ est $Z = 0$, donc $\frac{\partial F}{\partial X}([0 : 1 : 0]) = f = 0$ et $\frac{\partial F}{\partial Z}([0 : 1 : 0]) = g \neq 0$
- L'intersection de la tangente $Z = 0$ en $\mathcal{O}$ avec la courbe est donnée par l'équation $aX^3 + dX^2Y = 0$, pour avoir ensuite $\mathcal{O}$ point d'inflexion, il faut que ce point soit racine triple de $F(X : 1 : 0) = aX^3 + dX^2$, soit $d = 0$.
- Supposons $a = 0$, on se place alors dans le plan $Z = 0$. Donc :

$$\frac{\partial F}{\partial X} = 0 \quad \frac{\partial F}{\partial Y} = 0 \quad \frac{\partial F}{\partial Z} = eX^2 + jXY + gY^2$$

C'est un polynôme de degré 2 qui peut donc s'annuler. Quitte à perdre en généralité, on suppose donc $a \neq 0$ et la courbe reste elliptique.

On choisit alors un représentant de F ayant un coefficient 1 devant X^3 (possible car $a \neq 0$). Ainsi, $F(X, Y, Z) = X^3 + \alpha Z^3 + \beta X^2Z + \gamma Y^2Z + \delta Z^2X + \varepsilon Z^2Y + \zeta XYZ$; $\gamma \neq 0$.

On pose ensuite le changement de variables $Z' = -\frac{Z}{\gamma}$, on obtient F sous forme de Weierstrass. $\square$

3) Forme de Weierstrass réduite

Théorème 11 (Mise sous forme réduite de Weierstrass)

Si $\text{Car}(K) \neq 2, 3$, on peut mettre une courbe elliptique sous forme de Weierstrass réduite :

$$Y^2 Z = X^3 + aXZ^2 + bZ^3 \quad (1)$$

Démonstration. Soit E une courbe elliptique sous forme de Weierstrass, on pose d'abord, pour annuler le terme en XYZ :

$$X' = X, Y' = Y + \frac{a_1}{2}X, Z' = Z$$

Puis pour éliminer les termes en X^2 et Y :

$$X' = X + \frac{a_2}{3}, Y' = Y + \frac{a_3}{2}, Z' = Z$$

On arrive alors à la forme souhaitée, on note que ces changements de variables sont possibles grâce à l'hypothèse sur la caractéristique. $\square$

Corollaire 12 (Forme réduite affine)

En coordonnées non homogènes ($x = \frac{X}{Z}$ et $y = \frac{Y}{Z}$), on peut écrire cette équation :

$$E : y^2 = x^3 + ax + b \quad (2)$$

Ainsi que le point $\mathcal{O} = [0 : 1 : 0]$ qui est le seul point à l'infini.

On remarque que si P et Q sont deux points de E ont même abscisse, alors $Q = \pm P$.

Proposition 13 (Critère de singularité)

Soit E une courbe elliptique sous forme de Weierstrass, alors E est singulière si et seulement si la quantité $\Delta := 4a^3 + 27b^2$ est nulle.

Démonstration. E est une courbe de $\mathbb{P}^2(\mathbb{K})$ donnée par l'équation :

$$F(X, Y, Z) = Y^2 Z - X^3 - aXZ^2 - bZ^3$$

D'abord, $\frac{\partial F}{\partial Z}(\mathcal{O}) = 1 \neq 0$.

Passons maintenant en coordonnées affines :

$$E : f(x, y) = y^2 - x^3 - ax - b = 0$$

Si $P = (x_0, y_0)$ est un point singulier, alors $\frac{\partial f}{\partial y}(P) = 2y_0 = 0$, donc $y_0 = 0$ comme $\text{Car}(K) \neq 2$.

$\frac{\partial f}{\partial x}(P) = 3x_0^2 - a = 0$, donc $x_0^2 = \frac{a}{3}$. D'où $y_0^2 = 0 = x_0^3 + ax_0 + b = \frac{2}{3}ax_0 + b$.

On en déduit $x_0^2 = \frac{9b^2}{4a^2} = -\frac{a}{3}$. D'où $\Delta := 4a^3 + 27b^2 = 0$. $\square$

4) Structure de groupe abélien

a. Prémices

Proposition 14 (Intersections avec une droite)

Soient E une courbe elliptique et L une droite définies sur un corps $\mathbb{K}$.

Si E a au moins deux points d'intersection (comptés avec multiplicité) avec la droite L , alors E a exactement trois points d'intersection (comptés avec leur multiplicité) avec la droite L .

Démonstration. On suppose que la droite passant par P et Q est d'équation $y = \lambda x + \beta$.

Alors ces points vérifient $f(x, \lambda x + \beta) = 0$. C'est un polynôme de degré 3 admettant 2 racines, on peut ainsi le factoriser, ce qui prouve l'existence d'une troisième racine. $\square$

b. Approche géométrique

Définition 15 (Opposé)

Si $P = (x, y) \in E$, alors on définit l'opposé de P par $-P := (x, -y)$.

Définition 16 (Loi de la sécante-tangente)

On définit la loi $*$ dite de la **sécante-tangente** par :

- Si $P, Q \in E$ avec $P \neq Q$, alors $P * Q$ est le troisième point d'intersection de la droite passant par P et Q avec E .
- Si $P \in E$, alors $P * P$ est le troisième point d'intersection de la tangente à E en P .

Définition 17 (Loi de groupe)

On définit de plus la **loi additive** $P + Q := \mathcal{O} * (P * Q) = R$ où $-R$ est le troisième point d'intersection avec la droite.

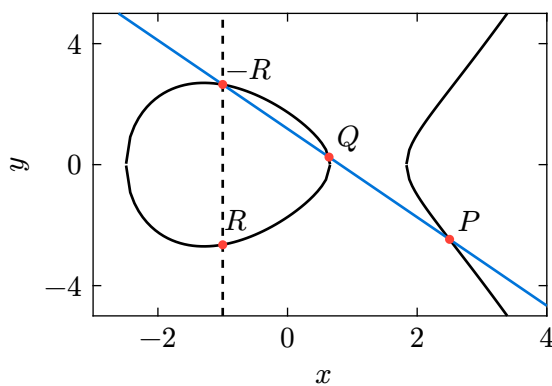


Fig. 3. – Calcul de $R = P + Q$ sur $E : y^2 = x^3 - 2x + 3$

c. Approche analytique

Proposition 18

Si P est un point de E , alors $P + \mathcal{O} = \mathcal{O} + P = P$. Cela fait de $\mathcal{O}$ l'élément neutre de E .

Proposition 19

Si $P = (x_1, y_1)$ et $Q = (x_2, y_2)$ sont deux points de E différents de $\mathcal{O}$, alors on définit $P + Q$ comme:

- Si $P = -Q$, alors $P + Q = P + (-P) = \mathcal{O}$
- Sinon, on pose $P + Q = (x_3, y_3)$ où $x_3 = \lambda^2 - x_1 - x_2$ et $y_3 = \lambda(x_1 - x_3) - y_1$, avec $\lambda = \begin{cases} \frac{3x_1^2 + a}{2y_1} & \text{si } P = Q \\ \frac{y_2 - y_1}{x_2 - x_1} & \text{si } P \neq Q \end{cases}$.

Démonstration. Si $P \neq Q$ et $P \neq -Q$, la sécante passant par P et Q a pour pente $\lambda = \frac{y_2 - y_1}{x_2 - x_1}$, et est donc d'équation $y = \lambda x + \delta$ où $\delta = y_1 - \lambda x_1$ est l'ordonnée à l'origine.

$$f(x, \lambda x + \delta) = -x^3 + \lambda^2 x^2 + (2\lambda\delta - a)x + (\delta^2 - b)$$

Donc par la formule de Viète, $\lambda^2 = x_1 + x_2 + x_3$, et $x_3 = \lambda^2 - x_1 - x_2$.

Dans le cas $P = Q$, on a la même preuve en prenant $\lambda = \frac{3x_1^2 + a}{2y_1}$ la pente de la tangente à E en P .

En effet, si $f(x) = x^3 + ax + b$, alors $2f'(x)f(x) = 3x^2 + a$, donc $f'(x_1) = \frac{3x_1^2 + a}{2y_1}$. $\square$

C'est justement la nécessité d'un inverse qui permettra plus tard de trouver un facteur premier.

Proposition 20

La loi $+$ est associative et commutative.

Théorème 21

$(E, +)$ a une structure de groupe abélien, avec $\mathcal{O}$ le neutre du groupe.

II - La méthode de factorisation de Lenstra

1) Lissité

Définition 22 (Lissité)

Si n est un entier et $n = \prod_{i=1}^m p_i^{\alpha_i}$ sa décomposition en facteurs premiers, on dit que n est **B -lisse** si pour tout $i \in \llbracket 1; m \rrbracket$, $p_i \leq B$.

On dit de même que n est **B -superlisse** lorsque pour tout $i \in \llbracket 1; m \rrbracket$, $p_i^{\alpha_i} \leq B$.

2) Algorithme de factorisation de Lenstra

Algorithme 1 (Algorithme de Lenstra)

Soient n un entier, et B une borne de lissité. On applique l'algorithme suivant pour trouver un facteur premier p de n à partir de courbes elliptiques $E_{a,b}$ de la forme :

$$E_{a,b} : y^2 = x^3 + ax + b \cup \mathcal{O}$$

- On vérifie que n n'est divisible ni par 2 ni par 3 (on a alors trouvé un diviseur premier de n), pour s'assurer que $\mathbb{F}_p$ sera de caractéristique différente de 2 ou 3, et que $E_{a,b}(\mathbb{F}_p)$ sera donc bien définie.
- On trouve tous les nombres premiers $p_1, \dots, p_k \leq B$.
- On choisit une courbe elliptique $E_{a,b}$ modulo n et un point $P \neq \mathcal{O}$ au hasard sur cette courbe.
- On calcule g le PGCD de $\Delta = 4a^3 + 27b^2$ et n .
- Si $g = n$, on change de courbe et de point.
- Sinon si $g > 1$, c'est un facteur non trivial de n que l'on renvoie.
- Sinon, on calcule pour chacun des p_i $e_i := \left\lfloor \frac{\log B}{\log p_i} \right\rfloor$ et $P \leftarrow p_i^{e_i} P$. Si ce calcul échoue, c'est que l'une des inversions nécessaires aux additions a échoué : on a trouvé un facteur.

- Si aucune exception n'est levée lors de ces calculs, on change de courbe et de point ou on abandonne la factorisation.

Il s'agit d'une variante de l'algorithme $p-1$ de Pollard exploitant les propriétés du corps fini $\mathbb{F}_p$.

Là où $p-1$ calcule des puissances, Lenstra calcule des produits, et on remplace les calculs explicites de PGCD par des calculs de pentes lors de l'addition de points.

Il peut être compliqué de trouver un point de manière aléatoire sur une courbe déjà choisie, on tire donc aléatoirement les coordonnées de P et on calcule ensuite b pour que P soit un point de $E_{a,b}$.

Théorème 23 (Choix optimal de B)

Soit n un entier avec un plus petit facteur premier p . Soit B la borne optimale pour trouver p à l'aide de l'algorithme de Lenstra.

On suppose qu'une courbe elliptique modulo p aléatoire de la forme $E_{a,b}$ où $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$ a un cardinal B -lisse avec probabilité u^{-u} où $u := \frac{\ln p}{\ln B}$.

Si $L(x) := \exp(\sqrt{\ln x} \cdot \ln \ln x)$, alors $B = L(p)^{\frac{\sqrt{2}}{2}}$.

De plus, le nombre total d'additions de points à effectuer pour trouver p est $L(p)^{\sqrt{2}}$, que l'on peut majorer pour tout facteur premier par $L(n)$.

Démonstration. L'équivalent asymptotique de la répartition des nombres premiers nous dit qu'il y a environ $\frac{B}{\ln B}$ premiers inférieurs à B . La plupart vérifie $p^2 > B$ donc l'exposant nécessaire dans Lenstra est en général 1.

La multiplication rapide nécessite environ $\log B$ additions. Donc un total de B calculs par courbe.

Par l'estimation précédente il faut donc essayer en moyenne u^u courbes.

On veut alors minimiser $f(B) = Bu^u$.

On prend a tel que $B = L(p)^a$ et on exprime nos termes en fonction de ce paramètre :

$$u \ln u \approx \frac{1}{a} \sqrt{\frac{\ln p}{\ln \ln p}} \frac{1}{2} \ln \ln p = \frac{1}{2a} \ln L(p)$$

Donc $f(B) \approx L(p)^{a + \frac{1}{2a}}$ où $L(p)$ est une constante positive pour $p \geq e^e$. Cette fonction est minimale pour $a = \frac{\sqrt{2}}{2}$.

Donc $B = L(p)^{\frac{\sqrt{2}}{2}}$ et le nombre estimé d'opérations est $f(B) = L(p)^{\sqrt{2}}$.

Comme $p \leq \sqrt{n}$ pour tout facteur premier p de n , on peut majorer par $L(N)$. $\square$

Dans la pratique, les valeurs de B sont tabulées.

III - Implémentation et résultats

1) Implémentation

J'ai choisi pour ce TIPE d'utiliser le langage C++ avec la librairie **GMP** afin de manipuler des entiers de taille non bornée, et ce afin de ne pas être limité par la taille initiale de 64 bits maximum.

J'ai ensuite opté pour les types suivants lors mon implémentation :

```
1 struct ellipticCurve {
2     mpz_class a;
3     mpz_class b;
4     mpz_class p;
5 }
```

```
1 struct ellipticPoint {
2     mpz_class x;
3     mpz_class y;
4     bool infty;
5     ellipticCurve *curve;
6 }
```

J'ai implémenté l'addition de points avec la même disjonction de cas que présentée, et la multiplication rapide de la même manière que l'exponentiation rapide sur les flottants.

J'ai utilisé le **crible d'Ératosthène** afin d'obtenir les premiers plus petits que B , cet algorithme nécessite un espace proportionnel à B ce qui le rend obsolète dès que notre facteur dépasse la cinquantaine de chiffres.

Le test de primalité est réalisé avec l'algorithme de **Miller-Rabin**, et les inversions modulaires par **l'algorithme d'Euclide étendu**.

2) Résultats

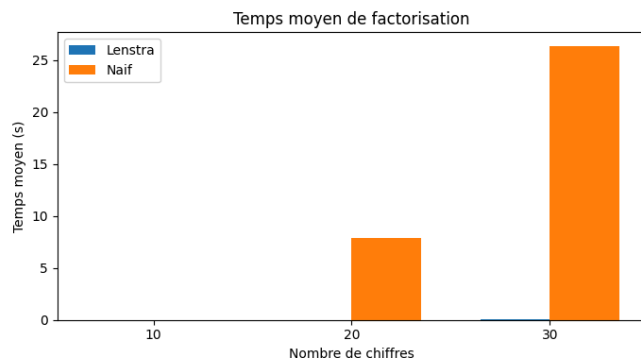


Fig. 4. – Comparaison des algorithmes naïfs et de Lenstra pour des nombres tirés aléatoirement

Remarque 24

L'algorithme naïf ne rendait pas en moins d'une heure pour plus de 30 chiffres.

D'autres tests ont été réalisés avec l'algorithme de Lenstra uniquement, en voici un :

```

1  n = 70990747356501633610903340638
2      49287869508897030027526772787
3
4  22:03:08 - Essai pour B = 11000
5  22:03:08 - Facteur: 219467308483
6
7  22:03:08 - Essai pour B = 50000
8
9  22:03:11 - Essai pour B = 250000
10
11 22:03:28 - Essai pour B = 1000000
12
13 22:06:19 - Essai pour B = 3000000
14
15 22:31:23 - Essai pour B = 11000000
16 22:32:16 - Facteur:
17      878948692609348221551
18 22:32:16 - Facteur:
19      36801742833925363771266239
20
21 =====
22 Temps d'exécution : 29:07.85 s
23 Mémoire : 18180 kb

```

Remarque 25

Le temps de calcul est plus élevé à cause de la taille des facteurs, pas de celle du produit directement.

IV - Conclusion

L'algorithme de Lenstra est utilisable en pratique pour factoriser des entiers ayant des facteurs ne dépassant pas une cinquantaine de chiffres mais devient inutilisable ensuite avec l'explosion de la borne de lissité et donc de l'espace mémoire disponible pour calculer les nombres premiers nécessaires.

Il s'agit de plus d'un algorithme facilement parallélisable en lançant de manière concurrente différents threads sur des courbes elliptiques différentes.

U - Bibliographie

1. SAMUEL S. WAGSTAFF, JR. : The Joy of Factoring : <https://www.ams.org/publications/authors/books/postpub/stml-68>, ISBN: 978-1-4704-1048-3
2. ALICE BOUILLET, DORIAN BERGER : Arithmétique des courbes elliptiques : https://perso.eleves.ens-rennes.fr/~aboui725/Stages/TER_M1.pdf
3. LAWRENCE C. WASHINGTON : Elliptic Curves Number Theory and Cryptography, Second Edition : CRC Press, Taylor & Francis Group, 2008, ISBN: 978-1-4200-7146-7
4. DIEGO IZQUIERDO : MAT562 : Introduction à la géométrie algébrique et courbes elliptiques : <https://synapses.polytechnique.fr/catalogue/2023-2024/ue/795/MAT562-introduction-a-la-geometrie-algebrique-et-courbes-elliptiques?from=D1>
5. JOSEPH H. SILVERMAN : The Arithmetic of Elliptic Curves : <https://link.springer.com/book/10.1007/978-0-387-09494-6>, ISBN: 978-0-387-09493-9

U1 - Annexes

1) Fonction de Dickman

La fonction de Dickman est une fonction permettant d'estimer le nombre d'entiers friables jusqu'à une certaine borne.

Elle est définie pour $u > 0$ par :

$$\rho(u) = \lim_{x \rightarrow +\infty} \frac{1}{x} \left| \left\{ 1 \leq n \leq x; P_1(n) \leq x^{\frac{1}{u}} \right\} \right|$$

Où $P_1(n)$ est le plus grand diviseur premier de n .

On peut en trouver une approximation lorsque u tend vers l'infini par le calcul :

$$\rho(u) \approx u^{-u}$$

C'est le modèle choisi pour le Théorème 23 car l'écart aux valeurs réels reste faible.

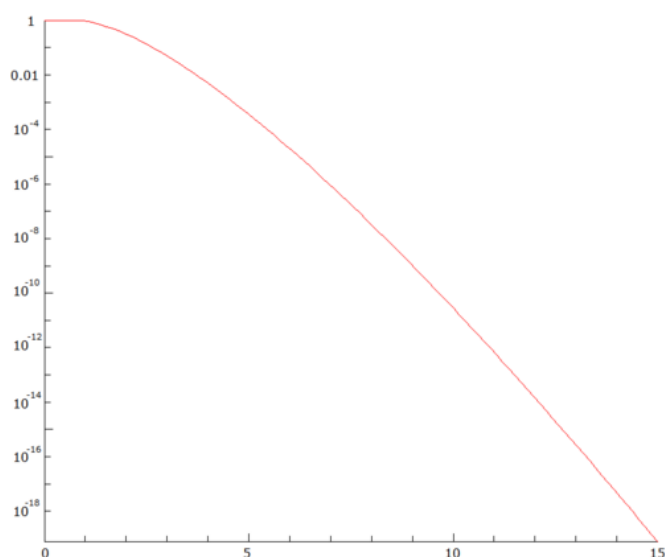


Fig. 5. – Tracé de $\rho(u)$ en fonction de u

Source : Wikipédia - Fonction de Dickman



Fig. 6. – Tracé de u^{-u} en fonction de u

2) Code

a. utils.hpp

```
1 #pragma once
```




```

2
3 #include <cstdint>
4 #include <gmpxx.h>
5
6 void hour(void);
7
8 void printFactor(mpz_class &p);
9 void printB(uint64_t B);

```

b. utils.cpp

```

1 #include "utils.hpp"
2 #include <iomanip>
3 #include <iostream>
4
5 void hour() {
6     // Obtenir le temps actuel
7     std::time_t now = time(nullptr);
8
9     // Convertir en heure locale
10    std::tm *localTime = std::localtime(&now);
11
12    std::cout << std::setfill('0') << std::setw(2) << localTime->tm_hour << ":"
13              << std::setfill('0') << std::setw(2) << localTime->tm_min << ":"
14              << std::setfill('0') << std::setw(2) << localTime->tm_sec << " - ";
15 }
16
17 void printFactor(mpz_class &p) {
18     hour();
19     std::cout << "\033[1;32mFacteur: " << p << "\033[1;0m\n";
20 }
21
22 void printB(uint64_t B) {
23     hour();
24     std::cout << "\033[1;34mEssai pour B = " << B << "\033[1;0m\n";
25 }

```

c. crible.cpp

```

1 #include <cmath>
2 #include <cstdint>
3
4 uint64_t *primeArray(const uint64_t n, uint64_t *c) {
5     if (n < 2) {
6         *c = 0;
7         return nullptr;
8     }
9
10    *c = 1;
11
12    uint64_t p = (n + 1) / 2 - 1; // Nombre d'impair entre 3 et n

```

```
13  bool *crible = new bool[p];
14
15  uint64_t fin = ((int)sqrt(n) - 3) / 2 + 1;
16
17  for (uint64_t i = 0; i < p; i++) {
18      crible[i] = true;
19  }
20
21  for (uint64_t i = 0; i < fin; i++) {
22      if (crible[i]) {
23          *c += 1;
24          uint64_t pas = 2 * i + 3;
25          uint64_t debut = (pas * pas - 3) / 2;
26
27          for (uint64_t j = debut; j < p; j += pas) {
28              crible[j] = false;
29          }
30      }
31  }
32
33  // Ajouter les termes après sqrt(n)
34  for (uint64_t i = fin; i < p; i++) {
35      if (crible[i]) {
36          *c += 1;
37      }
38  }
39
40  uint64_t *premiers = new uint64_t[*c];
41
42  premiers[0] = 2;
43  uint64_t j = 1;
44
45  for (uint64_t i = 0; i < p; i++) {
46      if (crible[i]) {
47          premiers[j++] = 2 * i + 3;
48      }
49  }
50
51  delete[] crible;
52  return premiers;
53 }
```

d. ellipticCurve.hpp

```
1  #pragma once
2
3  #include <gmpxx.h>
4  #include <iostream>
5  #include <ostream>
6
```



```

7  struct ellipticCurve {
8      mpz_class a;
9      mpz_class b;
10     mpz_class p;
11
12     ellipticCurve(const mpz_class &p, const std::pair<mpz_class, mpz_class> &P);
13
14     void fixCoeffs(const std::pair<mpz_class, mpz_class> &P);
15     mpz_class getDiscriminant();
16
17     friend std::ostream &operator<<(std::ostream &os, ellipticCurve &E);
18 };

```

e. ellipticCurve.cpp

```

1  #include "ellipticCurve.hpp" C++
2
3  ellipticCurve::ellipticCurve(const mpz_class &p,
4                               const std::pair<mpz_class, mpz_class> &P) {
5      this->a = rand() % p;
6      this->p = p;
7      fixCoeffs(P);
8  }
9
10 void ellipticCurve::fixCoeffs(const std::pair<mpz_class, mpz_class> &P) {
11     mpz_class x = P.first;
12     mpz_class y = P.second;
13     this->b = ((y * y) - (x * x * x) - (this->a * x)) % this->p;
14 }
15
16 mpz_class ellipticCurve::getDiscriminant() {
17     return (4 * (this->a * this->a * this->a) + 27 * (this->b * this->b)) %
18           this->p;
19 }
20
21 std::ostream &operator<<(std::ostream &os, ellipticCurve &E) {
22     os << "Paramètres de la courbe :\n- a = " << E.a << "\n- b = " << E.b
23       << "\n- p = " << E.p << "\n";
24     return os;
25 }

```

f. ellipticPoint.hpp

```

1  #pragma once C++
2
3  #include "ellipticCurve.hpp"
4  #include "gmpxx.h"
5  #include <ostream>
6
7  struct facteurTrouve : std::exception {
8     mpz_class facteur;

```

```

9   facteurTrouve(mpz_class facteur) : facteur(facteur) {};
10  const char *what() { return "Facteur trouvé par Lenstra"; }
11 };
12
13 struct ellipticPoint {
14     // Membres
15     mpz_class x;
16     mpz_class y;
17     bool infty;
18     ellipticCurve *curve;
19
20     // Constructeurs
21     ellipticPoint(const mpz_class &p);
22     ellipticPoint(bool infty, ellipticCurve &E);
23     ellipticPoint(const mpz_class &x, const mpz_class &y, ellipticCurve &E);
24
25     // Assignations :
26     ellipticPoint(const ellipticPoint &);
27     ellipticPoint &operator=(const ellipticPoint &);
28
29     // Egalite
30     bool operator==(const ellipticPoint &P) const;
31     bool operator!=(const ellipticPoint &P) const;
32
33     // Calculs
34     friend ellipticPoint operator+(const ellipticPoint &P1,
35                                   const ellipticPoint &P2);
36     friend ellipticPoint operator*(int k, const ellipticPoint &P);
37     friend ellipticPoint operator*(mpz_class k, const ellipticPoint &P);
38
39     // Utilitaire
40     void randomPointAndCurve(gmp_randclass &rng);
41
42     // Affichage
43     friend std::ostream &operator<<(std::ostream &os, ellipticPoint P);
44 };

```

g. ellipticPoint.cpp

```

1   #include "ellipticPoint.hpp"
2   #include "ellipticCurve.hpp"
3
4   // Constructeurs
5   ellipticPoint::ellipticPoint(const mpz_class &p) {
6       this->x = rand() % p;
7       this->y = rand() % p;
8       this->infty = false;
9   };
10
11  ellipticPoint::ellipticPoint(bool infty, ellipticCurve &E) {

```



```

12     this->x = 0;
13     this->y = 0;
14     this->infty = infty;
15     this->curve = &E;
16 }
17
18 ellipticPoint::ellipticPoint(const mpz_class &x, const mpz_class &y,
19                             ellipticCurve &E) {
20     this->x = x;
21     this->y = y;
22     this->infty = false;
23     this->curve = &E;
24 }
25
26 // Assignations : Ajouter += etc ????? => Remplacer + ?????
27 ellipticPoint::ellipticPoint(const ellipticPoint &) = default;
28 ellipticPoint &ellipticPoint::operator=(const ellipticPoint &) = default;
29
30 // Egalite
31 bool ellipticPoint::operator==(const ellipticPoint &P) const {
32     return (infty && P.infty) || (x == P.x && y == P.y);
33 };
34 bool ellipticPoint::operator!=(const ellipticPoint &P) const {
35     return !(*this == P);
36 }
37
38 // Calculs
39 ellipticPoint operator+(const ellipticPoint &P1, const ellipticPoint &P2) {
40     if (P1.curve != P2.curve)
41         throw std::runtime_error("Courbes différentes");
42
43     // Cas d'un point à l'infini
44     if (P1.infty)
45         return P2;
46     if (P2.infty)
47         return P1;
48
49     if ((P1.x == P2.x) && ((P1.y + P2.y) % P1.curve->p == 0)) // Points opposés
50         return ellipticPoint(true, *P1.curve);
51     else { // Supprimer duplicatas de code ici !!!!!
52         mpz_class s;
53         bool result;
54         mpz_class pgcd = 1;
55         if (P1 == P2) { // Un seul point
56             mpz_class num =
57                 3 * P1.x * P1.x + P1.curve->a; // Modulo sur ces deux lignes ?????
58             mpz_class denom = 2 * P1.y;
59
60             mpz_class inv;

```



```

61     result = mpz_invert(
62         inv.get_mpz_t(), denom.get_mpz_t(),
63         P1.curve->p.get_mpz_t()); // si result = 0, pas inversible !!!!!
64     if (!result)
65         mpz_gcd(pgcd.get_mpz_t(), denom.get_mpz_t(), P1.curve->p.get_mpz_t());
66     else
67         s = (num * inv) % P1.curve->p;
68 } else { // Deux points n'ayant pas même abscisse
69     mpz_class num = P2.y - P1.y;
70     mpz_class denom = P2.x - P1.x;
71
72     mpz_class inv;
73     result = mpz_invert(inv.get_mpz_t(), denom.get_mpz_t(),
74         P1.curve->p.get_mpz_t()); // Même remarque !!!!!
75     if (!result)
76         mpz_gcd(pgcd.get_mpz_t(), denom.get_mpz_t(), P1.curve->p.get_mpz_t());
77     else
78         s = (num * inv) % P1.curve->p;
79 }
80
81 if (1 < pgcd && P1.curve->p > pgcd) {
82     throw facteurTrouve(pgcd);
83 } else {
84     // Changer de courbe !!!!!
85 }
86
87 mpz_class x = (s * s - P1.x - P2.x) % P1.curve->p;
88 mpz_class y = (s * (P1.x - x) - P1.y) % P1.curve->p;
89 if (x < 0)
90     x += P1.curve->p;
91 if (y < 0)
92     y += P1.curve->p;
93 return ellipticPoint(x, y, *P1.curve);
94 }
95 };
96
97 ellipticPoint operator*(int k, const ellipticPoint &P) {
98     ellipticPoint Q = P;
99     ellipticPoint R = ellipticPoint(true, *P.curve);
100     while (k > 0) {
101         if (k & 1) { // k is odd
102             R = R + Q;
103         }
104         Q = Q + Q;
105         k >>= 1;
106     }
107     return R;
108 }
109

```

```

110 ellipticPoint operator*(mpz_class k, const ellipticPoint &P) {
111     ellipticPoint Q = P;
112     ellipticPoint R = ellipticPoint(true, *P.curve);
113     while (k > 0) {
114         if (mpz_tstbit(k.get_mpz_t(), 0)) { // k is odd
115             R = R + Q;
116         }
117         Q = Q + Q;
118         k >>= 1;
119     }
120     return R;
121 }
122
123 // Utilitaire
124 void ellipticPoint::randomPointAndCurve(gmp_randclass &rng) {
125     ellipticCurve *curve = this->curve;
126
127     this->x = rng.get_z_range(curve->p);
128     this->y = rng.get_z_range(curve->p);
129     curve->a = rng.get_z_range(curve->p);
130
131     curve->fixCoeffs(std::make_pair(this->x, this->y));
132 }
133
134 // Affichage
135 std::ostream &operator<<(std::ostream &os, ellipticPoint P) {
136     if (P.infty)
137         os << "Point à l'infini";
138     else
139         os << "(" << P.x << ", " << P.y << ")";
140     return os;
141 }

```

h. factor.hpp

```

1  #pragma once
2
3  #include <cstdint>
4  #include <gmpxx.h>
5
6  struct Param {
7      uint64_t maxChiffres;
8      uint64_t B;
9      uint64_t nbCourbesMax;
10 };
11
12 void trouverFacteur(const mpz_class &p, Param param, uint64_t *premiers,
13                    uint64_t c, mpz_class seed);
14
15 void factoriser(mpz_class &p, mpz_class *rngs, int NB_THREADS);

```



i. factor.cpp

```

1  #include "factor.hpp"
2  #include "ellipticPoint.hpp"
3  #include "utils.hpp"
4
5  #include <cmath>
6  #include <cstdint>
7  #include <gmpxx.h>
8  #include <mutex>
9  #include <queue>
10 #include <thread>
11 #include <vector>
12
13 extern uint64_t *primeArray(const uint64_t n, uint64_t *c);
14
15 const std::vector<Param> PARAMETRES_OPTIMAUX = {
16     {20, 11000, 74}, {25, 50000, 221}, {30, 250000, 453},
17     {35, 1000000, 984}, {40, 3000000, 2541}, {45, 11000000, 4949},
18     {50, 43000000, 8266}, {55, 110000000, 20158}, {60, 260000000, 47173},
19     {65, 850000000, 77666}};
20
21 mpz_class facteur = 1;
22 uint64_t nbCourbes = 0;
23 std::mutex verrou;
24
25 void setFacteur(mpz_class &p) {
26     verrou.lock();
27     facteur = p;
28     verrou.unlock();
29 }
30
31 void trouverFacteur(const mpz_class &p, Param param, uint64_t *premiers,
32                    uint64_t c, mpz_class seed) {
33     gmp_randclass rng(gmp_randinit_default);
34     rng.seed(seed ^ time(NULL));
35
36     uint64_t B = param.B;
37     uint64_t nbCourbesMax = param.nbCourbesMax;
38
39     mpz_class delta, pgcd;
40
41     ellipticPoint P(p);
42     ellipticCurve E(p, std::make_pair(P.x, P.y));
43     P.curve = &E;
44
45     bool ok = true;
46
47     while (ok) {
48         delta = E.getDiscriminant();

```

```

49     mpz_gcd(pgcd.get_mpz_t(), delta.get_mpz_t(), E.p.get_mpz_t());
50
51     while (pgcd != 1) {
52         if (pgcd < p)
53             setFacteur(pgcd);
54         P.randomPointAndCurve(rng);
55         delta = E.getDiscriminant();
56         mpz_gcd(pgcd.get_mpz_t(), delta.get_mpz_t(), E.p.get_mpz_t());
57     }
58
59     try {
60         for (uint64_t i = 0; i < c && premiers[i] <= B; i++) {
61             mpz_class e = log(B) / log(premiers[i]);
62             mpz_class result;
63             mpz_class premier = premiers[i];
64             mpz_powm(result.get_mpz_t(), premier.get_mpz_t(), e.get_mpz_t(),
65                     p.get_mpz_t());
66
67             P = result * P;
68         }
69     } catch (const facteurTrouve &error) {
70         mpz_gcd(pgcd.get_mpz_t(), error.facteur.get_mpz_t(), E.p.get_mpz_t());
71
72         if (pgcd > 1 && pgcd < p)
73             setFacteur(pgcd);
74     }
75
76     verrou.lock();
77     nbCourbes++;
78     ok = nbCourbes < nbCourbesMax && facteur == 1;
79     verrou.unlock();
80 }
81 }
82
83 void factoriser(mpz_class &p, mpz_class *rngs, int NB_THREADS) {
84     // Affichage initial
85     hour();
86     std::cout << "\033[1;32mÀ factoriser: " << p << "\033[0m\n\n";
87
88     // Threads
89     std::thread *threads = new std::thread[NB_THREADS];
90
91     // Initialisation des paramètres
92     Param param = PARAMETRES_OPTIMAUX.front();
93     uint64_t currentParametre = 0;
94
95     // Crible initial
96     uint64_t c;
97     uint64_t *primes = primeArray(param.B, &c);

```

```
98     printB(param.B);
99
100    mpz_class n = 0;
101    while (p % 2 == 0) {
102        n = 2;
103        printFactor(n);
104        p /= 2;
105    }
106    while (p % 3 == 0) {
107        n = 3;
108        printFactor(n);
109        p /= 3;
110    }
111
112    // File à factoriser
113    std::queue<mpz_class> remaining;
114    remaining.push(p);
115
116    while (!remaining.empty()) {
117        mpz_class p = remaining.front();
118        if (mpz_probab_prime_p(p.get_mpz_t(), 20)) {
119            remaining.pop();
120            printFactor(p);
121        } else {
122            for (int i = 0; i < NB_THREADS; i++) {
123                threads[i] = std::thread(trouverFacteur, p, param, primes, c, rngs[i]);
124            }
125            for (int i = 0; i < NB_THREADS; i++) {
126                threads[i].join();
127            }
128
129            if (facteur == 1) {
130                if (currentParametre < PARAMETRES_OPTIMAUX.size()) {
131                    currentParametre++;
132                    param = PARAMETRES_OPTIMAUX[currentParametre];
133                    primes = primeArray(param.B, &c);
134                }
135
136                std::cout << "\n";
137                printB(param.B);
138            } else {
139                remaining.pop();
140                remaining.push(facteur);
141                remaining.push(p / facteur);
142            }
143
144            facteur = 1;
145        }
146    }
```

```
147  
148  delete[] primes;  
149 }
```